

Práctica 3 – Reconocimiento de Escenas con modelos Bag-of-Words

González Sánchez, Iker – López Martínez, Alejandro

1 PREGUNTAS TAREAS OPCIONALES

3.1. Cree un esquema de clasificación de imágenes con los siguientes detalles: + Características: HOG con parámetro tam=100 + Modelo: BOW con max_iter=10 + Clasificador: KNN con K=5 + Otros: Ratio train_test=0.20, y Máx número datos por categoría: 200 Debe utilizar las funciones para obtener features en p3_tarea2.py, funciones

construir_vocabulario/obtener_bags_of_words (fichero p3_tarea1.py), y función load_image_dataset (fichero p3_utils.pyc). Para construir este esquema puede basarse en sklearn.model_selection.train_test_split, sklearn.neighbors.KNeighborsClassifier Y aplíquelo sobre el dataset scene15 disponible en el material de la práctica con una partición determinista. A continuación, responda a las siguientes preguntas:

3.1.1. Compare los resultados obtenidos al utilizar las características HOG y Tiny (con parámetro tam=64), en términos de rendimiento de clasificación sobre los datos de entrenamiento y test. Utilice valores por defecto indicados en el enunciado con un tamaño de diccionario de 50.

Como se observa en la tabla 2, los resultados indican que los descriptores HOG ofrecieron un rendimiento significativamente superior, con una precisión del 60.12% en entrenamiento y del 41.00% en prueba, en contraste con los descriptores Tiny, que lograron un 18.54% y 16.33%, respectivamente. Esto sugiere que HOG, al capturar texturas y patrones detallados de las imágenes, es más eficaz para tareas como la clasificación de escenas, mientras que Tiny, debido a su simplificación extrema de la información, resulta menos adecuado para este tipo de problemas.

3.1.2. Varíe el tamaño del diccionario BOW (hasta un valor máximo de 200) y estudie el rendimiento de clasificación sobre los datos de entrenamiento y test. Utilice valores por defecto indicados en el enunciado.

Como se observa en la figura 1, la precisión en el conjunto de entrenamiento muestra una mejora inicial conforme se

incrementa el tamaño del vocabulario, estabilizándose en torno al 65% con vocabularios más grandes. Esto indica que un vocabulario más amplio permite capturar de manera más efectiva las características del conjunto de datos, maximizando la representación del mismo.

Por otro lado, la precisión en el conjunto de prueba también mejora de manera progresiva, estabilizándose alrededor del 45%. Este comportamiento sugiere que vocabularios pequeños no son suficientes para generalizar adecuadamente, mientras que tamaños mayores logran una mejor representación de las características relevantes. Sin embargo, no se han observado mejoras significativas al incrementar el vocabulario más allá de 200 palabras, lo que permite concluir que un rango entre 100 y 200 es el tamaño óptimo, equilibrando generalización y eficiencia.

3.1.3. Varíe el número de vecinos del clasificador KNN (hasta 21) con HOG y estudie el rendimiento de clasificación sobre los datos de entrenamiento y test. Muestre resultados visuales de aciertos/errores (se recomienda la función create_webpage_results). Utilice valores por defecto indicados en el enunciado con el tamaño de diccionario BOW óptimo obtenido en la pregunta 3.1.2.

Como se observa en la figura 2, la precisión en el conjunto de entrenamiento disminuye de manera pronunciada a medida que aumenta el valor de k. Con valores bajos, como 1 o 3, el modelo se ajusta excesivamente a los datos de entrenamiento, alcanzando una precisión cercana al 100% debido al sobreajuste. Sin embargo, al incrementar k, el modelo se vuelve más robusto al ruido, lo que reduce este sobreajuste.

Por otro lado, la precisión en el conjunto de prueba muestra una ligera mejora inicial con valores crecientes de k, estabilizándose alrededor del 45%-50%. Este comportamiento sugiere que un valor moderado de k permite al modelo generalizar mejor, evitando tanto el sobreajuste como el subajuste. Como conclusión, se identifica que el rango óptimo para k está entre 15 y 21, donde el modelo logra un balance adecuado entre la precisión en los conjuntos de entrenamiento y prueba. Esto demuestra que un número más alto de vecinos contribuye a una predicción más estable y generalizada.

3.2. Cree un esquema de clasificación de imágenes con los siguientes detalles:

+ Características: HOG con parámetro tam=100

+ Modelo: BOW con max_iter=10

+ Clasificador: SVM (lineal)

+ Otros: Ratio train_test=0.20, y Máx número datos por categoría: 200

Debe utilizar las funciones para obtener features en p3_tarea2.py, funciones

construir_vocabulario/obtener_bags_of_words (fichero p3_tarea1.py), y

función load_image_dataset (fichero p3_utils.pyc).

Para construir este esquema

puede basarse en

sklearn.model_selection.train_test_Split, sklearn.svm

Y aplíquelo sobre el dataset scene15 disponible en el material de la práctica con una

partición determinista. A continuación, responda a las dos siguientes preguntas:

3.2.1 Varíe el tamaño del diccionario BOW (hasta un valor máximo de 200) y estudie como varía el rendimiento de clasificación sobre los datos de entrenamiento y test. Utilice valores por defecto indicados en el enunciado.

Como se observa en la Figura 3, a medida que se incrementa el tamaño del vocabulario, la precisión en los datos de entrenamiento mejora significativamente, alcanzando casi el 100% con vocabularios grandes. Esto indica que el modelo es capaz de memorizar los datos de entrenamiento cuando se dispone de un mayor número de palabras en el vocabulario. Sin embargo, la precisión en los datos de prueba se mantiene prácticamente constante alrededor del 50%, con fluctuaciones menores. Esto sugiere que el modelo no generaliza bien a datos no vistos, probablemente debido a un sobreajuste a los datos de entrenamiento.

3.2.2 Investigue y compare los distintos tipos de kernels no lineales para clasificadores

SVM (e.g., rbf y poly) sobre los datos de entrenamiento y test. Razone por-que se obtiene unos resultados distintos a la pregunta anterior con una SVM lineal. Posteriormente, analice el rendimiento de la configuración con el mejor resultado y muestre resultados visuales de aciertos/errores (se recomienda la función crea-te_webpage_results).

Utilice valores por defecto indicados en el enunciado con el tamaño de diccionario BOW óptimo obtenido en la pregunta 3.2.1.

La figura 4, muestra el rendimiento de clasificación usando un kernel RBF. La precisión en los datos de entrenamiento aumenta progresivamente con el tamaño del vocabulario, estabilizándose en valores cercanos al 95% para vocabularios grandes. Esto indica que el kernel

RBF es muy efectivo para capturar patrones en los datos de entrenamiento. Por otro lado, la precisión en los datos de prueba muestra un incremento inicial hasta un máximo alrededor del 65%, pero después se estabiliza sin alcanzar valores más altos, a pesar de que el tamaño del vocabulario sigue aumentando. Esto sugiere que el kernel RBF permite un mejor equilibrio entre aprendizaje y generalización en comparación con el kernel lineal. Sin embargo, el modelo podría seguir enfrentando cierto grado de sobreajuste.

En la figura 5, la precisión en los datos de entrenamiento también muestra un aumento progresivo a medida que crece el tamaño del vocabulario, alcanzando valores cercanos al 90%. Este comportamiento refleja que el modelo aprende cada vez mejor los datos de entrenamiento, aunque no tan eficientemente como con el kernel RBF. La precisión en los datos de prueba presenta un patrón distinto: comienza en aproximadamente un 50%, mejora ligeramente al aumentar el tamaño del vocabulario, pero luego disminuye con vocabularios más grandes. Este comportamiento sugiere que el kernel polinomial es menos robusto en términos de generalización y tiende a sobreajustarse más rápidamente, especialmente con vocabularios grandes.

En comparación con el kernel lineal (gráfica anterior), los kernels no lineales (RBF y polinomial) presentan una mayor capacidad para capturar relaciones complejas en los datos. El kernel RBF, en particular, destaca por lograr mejores precisiones en el conjunto de prueba en relación con los otros kernels. Por el contrario, el kernel polinomial parece más susceptible al sobreajuste, lo que explica la caída en la precisión de prueba con vocabularios grandes. Estos resultados distintos se deben a las características propias de los kernels no lineales, que permiten modelar mejor las relaciones no lineales en los datos, mientras que el kernel lineal es limitado a relaciones estrictamente lineales. Sin embargo, los modelos no lineales también son más propensos al sobreajuste si no se ajustan correctamente los parámetros.

3.3. Cree un esquema de clasificación de imágenes con los siguientes detalles:

+ Características: HOG con parámetro tam=100

+ Modelo: BOW con max_iter=10

+ Clasificador: Random Forest

+ Otros: Ratio train_test=0.20, y Máx número datos por categoría: 200

Debe utilizar las funciones para obtener features en p3_tarea2.py, funciones

construir_vocabulario/obtener_bags_of_words

(fichero p3_tarea1.py), y

función load_image_dataset (fichero p3_utils.pyc).

Para construir este esquema

puede

basarse

en

sklearn.model_selection.train_test_Split,

sklearn.ensemble.RandomForestClassifier

Y aplíquelo sobre el dataset scene15 disponible en el material de la práctica con una

partición determinista. A continuación, responda a

las dos siguientes preguntas:

3.3.1Investigue la función `sklearn.ensemble.RandomForestClassifier` y seleccione solo uno de los parámetros disponibles. Compare y razone los resultados para distintos valores del parámetro seleccionado. Posteriormente, analice el rendimiento de la configuración con el mejor resultado y muestre resultados visuales de aciertos/errores (se recomienda la función `create_webpage_results`). Utilice valores por defecto indicados en el enunciado con el tamaño de diccionario BOW óptimo obtenido en la pregunta 3.2.1.

La imagen 6, muestra que a medida que aumenta el número de estimadores, la precisión del modelo de entrenamiento se mantiene relativamente constante alrededor del 95-100%. Sin embargo, la precisión del modelo de prueba aumenta gradualmente a medida que se incrementa el número de estimadores, empezando alrededor del 50% y alcanzando aproximadamente el 60-65% cuando el número de estimadores es igual o superior a 1500.

Este comportamiento sugiere que el modelo de Random Forest se beneficia de un mayor número de estimadores, logrando una mejor generalización y precisión en los datos de prueba, sin caer en un sobreajuste excesivo en los datos de entrenamiento. La estabilización de la precisión del modelo de entrenamiento también indica que el modelo ha alcanzado un nivel de complejidad adecuado para capturar las características relevantes de los datos.

Tarea	Horas dedicadas (Alejandro López)	Horas dedicadas (Iker González)
Tarea 1	3,5	3,5
Tarea 2	2	
Memoria	5,5	6

Tabla 1. Tabla horas de trabajo

Descriptor	Precisión Entrenamiento (%)	Precisión Test (%)
Tiny	18.54	16.33
HOG	60.12	41.00

Tabla 2. Comparación de precisiones ej 3.1.1

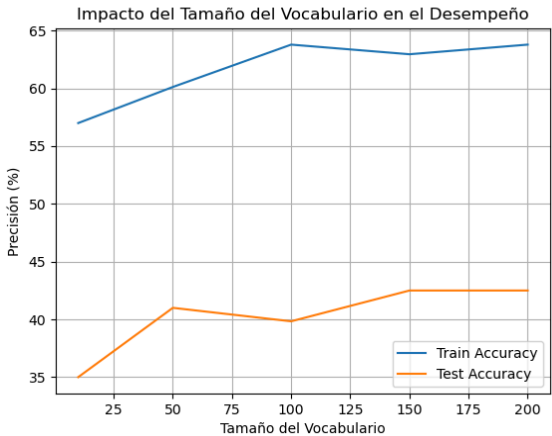


Fig. 1. Impacto del tamaño del vocabulario.

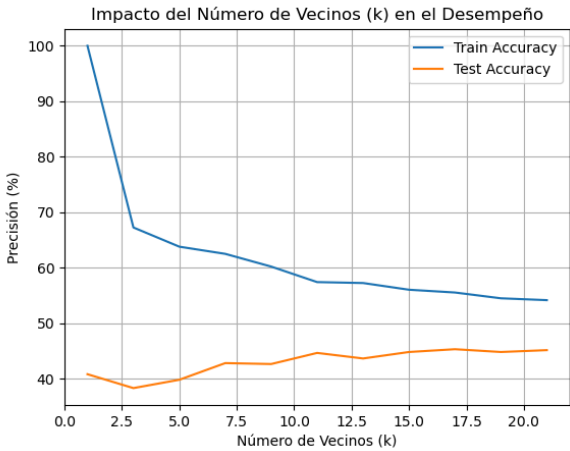


Fig. 2. Impacto del número de vecinos.

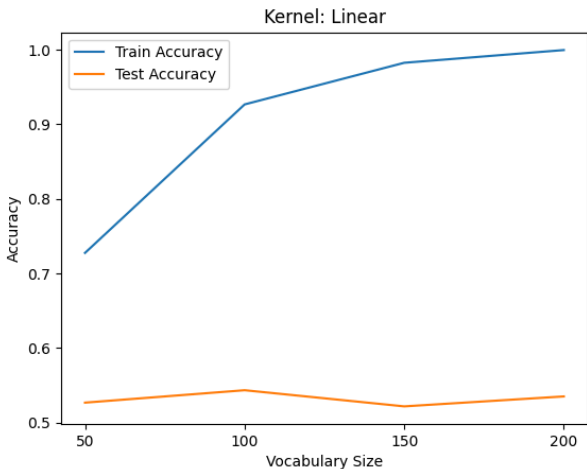


Fig. 3. Resultados obtenidos utilizando un Kernel lineal.

observar en la Tabla 1.

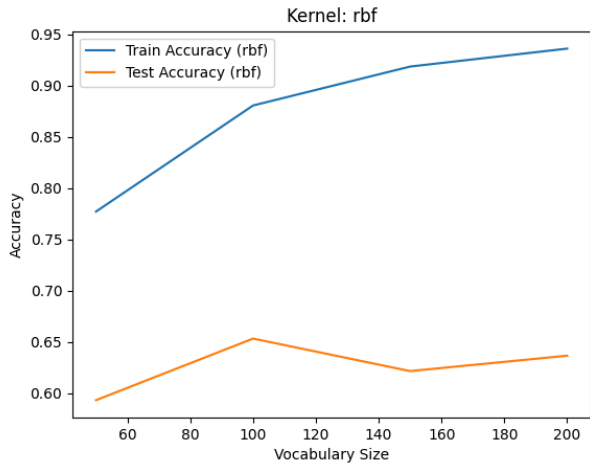


Fig. 4. Resultados obtenidos utilizando un Kernel no lineal RBF.

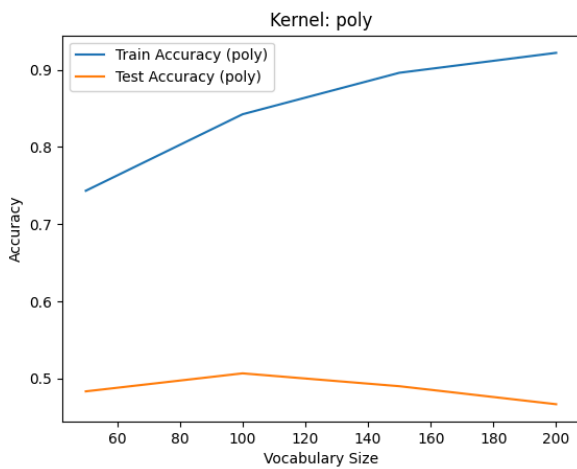


Fig. 5. Resultados obtenidos utilizando un Kernel no lineal polinomial.

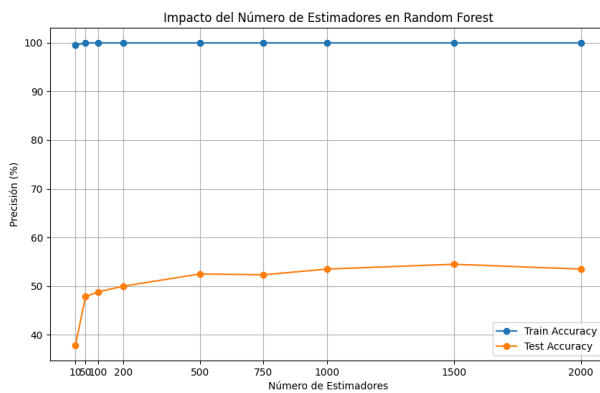


Fig. 6. Resultados que muestran el impacto del número de estimadores en un modelo de Random Forest.

4 CARGA DE TRABAJO

La carga (en horas) de cada tarea de esta práctica se puede